

ROBOTICS · LESSON 10

Go There

Fuse distance and heading into a live (x, y, θ) pose and drive to a goal you name — avoiding obstacles, reporting live. The whole track, working at once.

ROBO-010 · ~55 MIN · INTERMEDIATE

1. The one idea

Pose. Combine distance (odometry, Lesson 7) and heading (the IMU, Lesson 9) into a live estimate of where the robot is — its (x, y, θ) — then drive it to a goal you name, avoiding obstacles on the way and reporting as it goes. The robot knows where it is and goes where you point.

2. Why it matters

Every piece of the track converges here. Actuation (L1), the safety guard (L2, L5), sensing (L3), locomotion (L4), feedback (L6), distance (L7), visibility (L8), and heading (L9) — a navigating robot is all of them at once. Tracking pose and driving toward a goal is the foundation of mobile robotics: "go to (x, y) " is what a delivery robot, a vacuum, and a planetary rover all do. This is where the whole course lands.

3. Parts & wiring

Nothing new — everything from the track, assembled: the rover (L4) with wheel encoders (L6/L7), the ultrasonic sensor and its `SAFE_CM` guard (L3/L5), and the MPU-6050 for heading (L9). One robot, all of it wired at once.

4. Predict (do this before uploading)

- Place the robot at $(0, 0)$ facing +x and command `goTo(100, 0)` then `goTo(100, 100)`. Does it land on the mark?
- Drive a square back to $(0, 0)$. How far from "home" does it actually stop?
- Put a box on the straight-line path to the goal. What should the robot do?

5. Build it — estimate, aim, step, repeat

```
// Assembled from the track: TICKS_PER_CM + encoders (L7), heading (L9),
// drive() (L4), readDistanceCm() + SAFE_CM + turnAway() (L5), turnTo() (L9).
float x = 0, y = 0;           // position in cm, in the start frame
extern float heading;         // degrees, from Lesson 9

// call each step with the average wheel ticks rolled since the last call
void updatePose(long dTicks) {
    float d = dTicks / TICKS_PER_CM;
    float th = heading * PI / 180.0;
    x += d * cos(th);          // dead reckoning: project travel onto the heading
    y += d * sin(th);
}

void goTo(float tx, float ty, int speed) {
    while (true) {
        float dx = tx - x, dy = ty - y;
        if (sqrt(dx * dx + dy * dy) < 3.0) break;          // arrived (within 3 cm)

        long cm = readDistanceCm();
        if (cm > 0 && cm < SAFE_CM) {                      // the guard, riding along
            drive(0, 0, 0);
            turnAway();                                     // avoid, don't ram the goal
            continue;
        }
        turnTo(atan2(dy, dx) * 180.0 / PI);                // face the goal
        driveStep(speed);                                   // roll forward a bit, then
        updatePose()
    }
    drive(0, 0, 0);                                         // stop at the goal
}
```

The shape is the whole course in one loop: **estimate the pose, aim at the goal, step toward it, let the guard override for obstacles, repeat until you arrive.** Every subsystem you built is in those lines — and `updatePose` is the heart: it projects how far you rolled onto the direction you faced, accumulating an `(x, y)` estimate.

6. Test against your prediction

Mark `(0, 0)` on the floor, run `goTo(100, 0)` then `goTo(100, 100)`, and measure where it lands. Set a box on the path and watch the guard stop the robot short of its goal instead of driving through it. Then run a full square back to `(0, 0)` and measure the gap to home — that gap is the accumulated drift, made visible on the floor.

7. What goes wrong (pre-loaded debugging)

- **Lands consistently off the mark** → odometry or heading calibration. Re-tune `TICKS_PER_CM` (L7) and the gyro bias (L9).
- **The error grows along the path** → dead-reckoning drift compounds — expected, because there is no external correction yet. That's the point of the next frontier.
- **Spins hunting for the bearing** → `turnTo` gains too high or heading too noisy; damp the turn.
- **Drives through an obstacle to reach the goal** → the guard check is too slow or `SAFE_CM` too small for the speed — the Lesson 5 lesson, again.
- **"Home" is a long way off after a big loop** → the honest wall: pure dead reckoning can't close large loops. Real robots fix this against the world.

8. Checkpoint

The robot tracks its pose and drives to a goal you name, avoiding obstacles and reporting live — every piece of the track working together. That is mobile robotics in miniature: sense, estimate, decide under a guard, act, repeat. It drifts, and seeing exactly how and why is what points you at the real frontier — correcting the estimate against the world. You've built the whole loop; everything past here is making the estimate better.

9. Push further

- Patrol a list of waypoints and watch the drift accumulate — a vivid case for closing the loop on the world.
- Snap the pose to a known landmark when you see it (a wall at a known x) — the seed of SLAM.
- Stream `(x, y)` to the Lesson 8 telemetry page and watch the robot's path trace itself live.

That completes the Robotics track: **actuate, guard, sense, locomote, close the loop, control, place, make visible, orient, and navigate**. The next frontier — building a map and localizing in it (SLAM) so the pose stops drifting — is named here, not taught. You have the whole loop; that frontier is the loop, done better.

